UC San Diego

Self-Evolving Game Agent with MCTS and Tool-Integration

CSE 291A - AI Agents - Team 9

Jason Zhu, Yinuo Zhou, Hongrui (Frank) Zhang, Yaran Yang, Zavier Zhao, Fengfei Yu

University of California, San Diego | Mar 12th, 2026

Introduction

Motivation & Core Problems

- Solving turn-based games as MDP problems has been an effective strategy (Go, etc.)
- One method is **Monte Carlo Tree Search**. it searches the game environment and finds promising paths algorithmically.
- AlphaZero style MCTS **trains deep neural networks** for state value estimations, which is hard to setup right and has massive entry cost.
- Earlier MCTS+LLM work focused on **online, step-wise LLM** guidances, which is costly and constrained by static local views.
- Emergence of LLM skills/mcp gave us motivation for another direction: **policy improvement with rule based heuristics proposed autonomously by LLM.**

Introduction

- We developed a **LLM tool-evolving agent** for **turn-based games:**
  - Interacts with different game environments (Sokoban, TextWorld, etc.)
  - Collect game traces, analyze feedback, do **task proposal**, generate pluggable heuristics, improve upon successful validation. Iterate.
  - Mainly used **Monte Carlo Tree Search** as a foundational framework, but also explored other ideas like **A*** for games our MCTS struggled.
  - **Generates heuristics** for MCTS, saw improvement even with low iterations.
  - LLM acts as an **offline coding agent**, while code based heuristics run online efficiently. Contrast with using LLM as an online decision agent or training deep neural networks.
  - Generated heuristics are **human understandable**, code artifacts instead of weights.

Audience of Interest

- **LLM users and researchers:**
  - We explore the application of **LLM tool creation**, a forefront research direction
  - We explore **self-evolving** agent paradigm, populating and optimizing a tool collection
  - **Task proposal** - we explored LLM's ability to learn from environment feedback and propose improvements autonomously, alike human players' improvement from experience.
- **Game players:**
  - Generating heuristics offline as tools is **less expensive**, can lead to efficient gameplay improvements for solving complex games without large entry cost.

Related Work

Keywords

Games

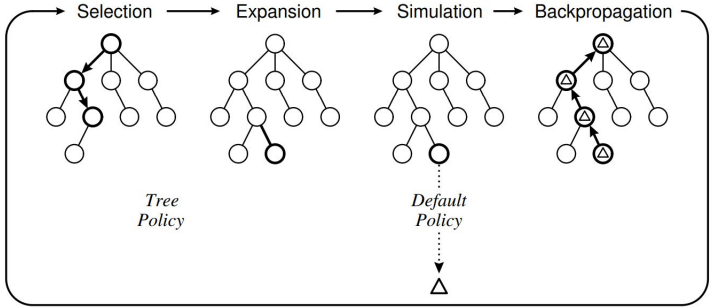
LLM Task Proposal

Tool-creati on Agent System

Monte Carlo Tree Search (MCTS)

A Survey of Monte Carlo Tree Search Methods
[2012] Cameron Browne et. al

- **Challenge:** How to find optimal decisions in given domain (complex game as MDP)?
- **Paper:** Maps problem to search tree, perform 4 stages (selection, exploration, simulation, back propagation) with tree policy to understand game states and identify optimal decisions
- Exploration heuristics like UCT balances exploration and exploitation with simple data
- **Limitation:** Lacks reasoning over environment data, constraint by limited information
- **Our learning:** Solve complex games base on a MCTS framework
- **Our approach:** Use LLM to enhance reasoning over environment and state information



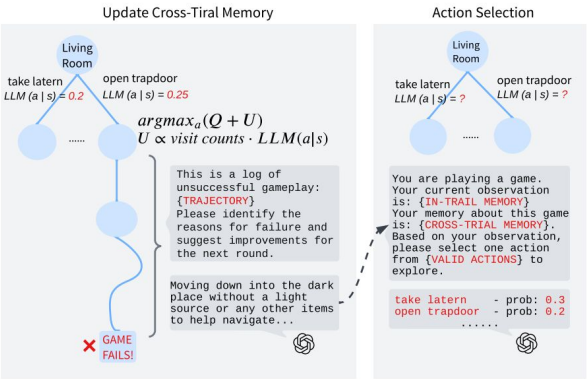
UCT only knows reward and visit counts

$$\textbf{return} \quad \underset{v' \in \text{children of } v}{\arg\max} \frac{Q(v')}{N(v')} + c\sqrt{\frac{2 \ln N(v)}{N(v')}}$$

MC-DML (Monte-Carlo planning with Dynamic Memory-guided Large language model)

[ICLR 2025] Zijing Shi et. al

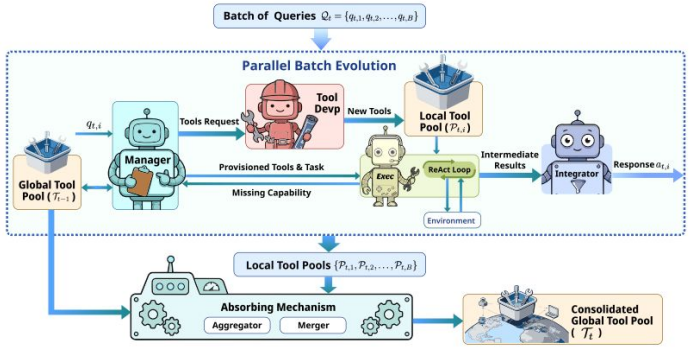
- **Challenge:** Past MCTS+RL limitations, like uncertain exploration and lack of reasoning
- **Paper:** Deploy LLM as policy (action decision) maker in text games, enrich context with dynamic memory mechanism (in-track, cross-track memory)
- **Limitation:** High cost, confusion with local memory management
- **Our learning:** Utilize LLM's reasoning capabilities and memory management
- **Our approach:** We generate heuristics, used for many steps; we study summarized traces in a global point of view, tool generation



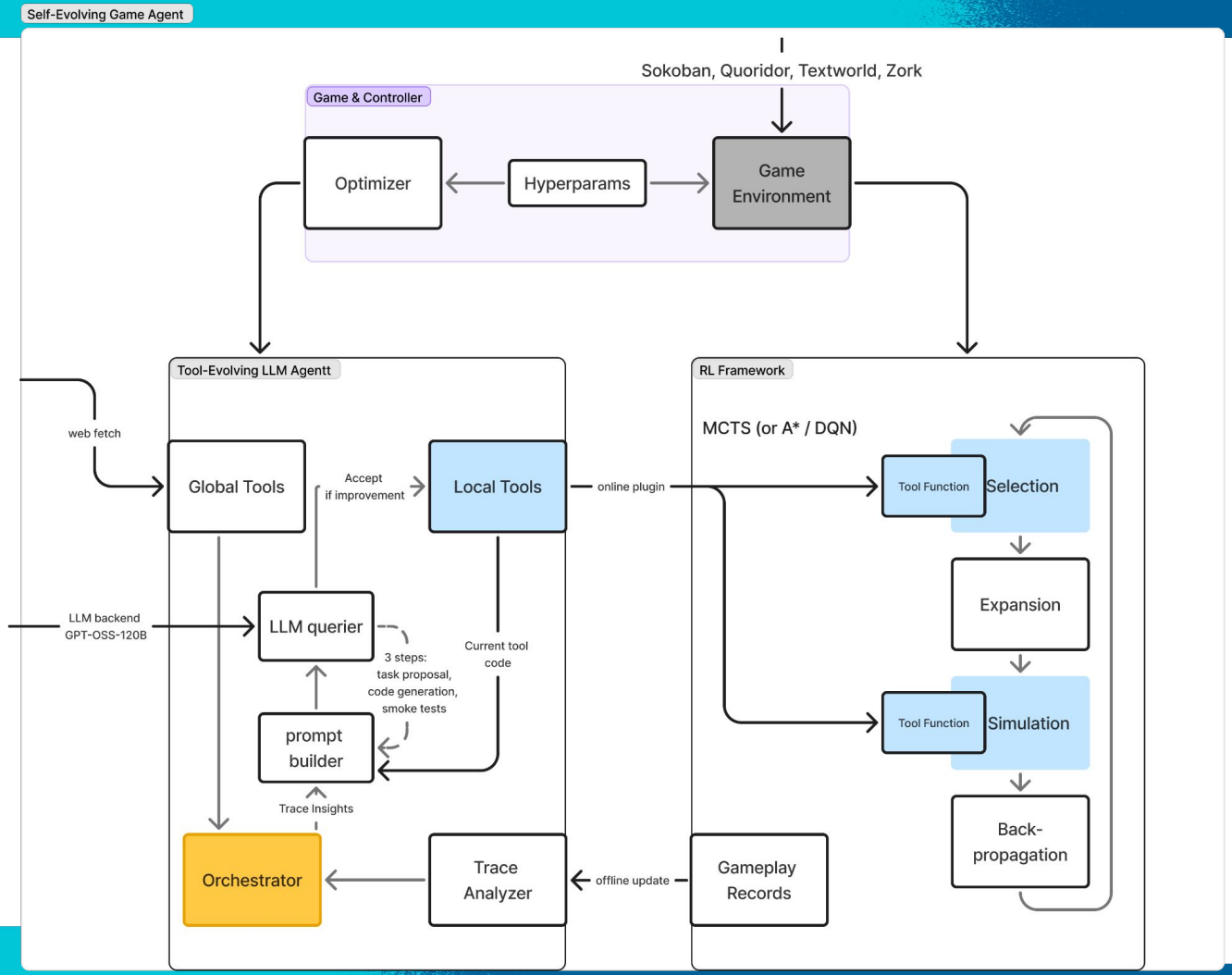
Yunjue Agent Tech Report: A Fully Reproducible, Zero-Start In-Situ Self-Evolving Agent System for Open-Ended Tasks

[2026] Haotian Li et. al

- **Challenge:** LLM requires human supervision to solve hard problems
- **Paper:** Multi-agent architecture that solves sequence of real-world problems by generating, synthesizing, optimizing, and reusing tools (python scripts) in parallel
- **Limitation:** Focuses on deterministic problems, can benefit from targeted workflow
- **Our learning:** Novel tool generation paradigm reduces supervision in our system
- **Our approach:** Using LLM to analyze, generate, and improve MCTS or other heuristics as tools for validate agentic propose task



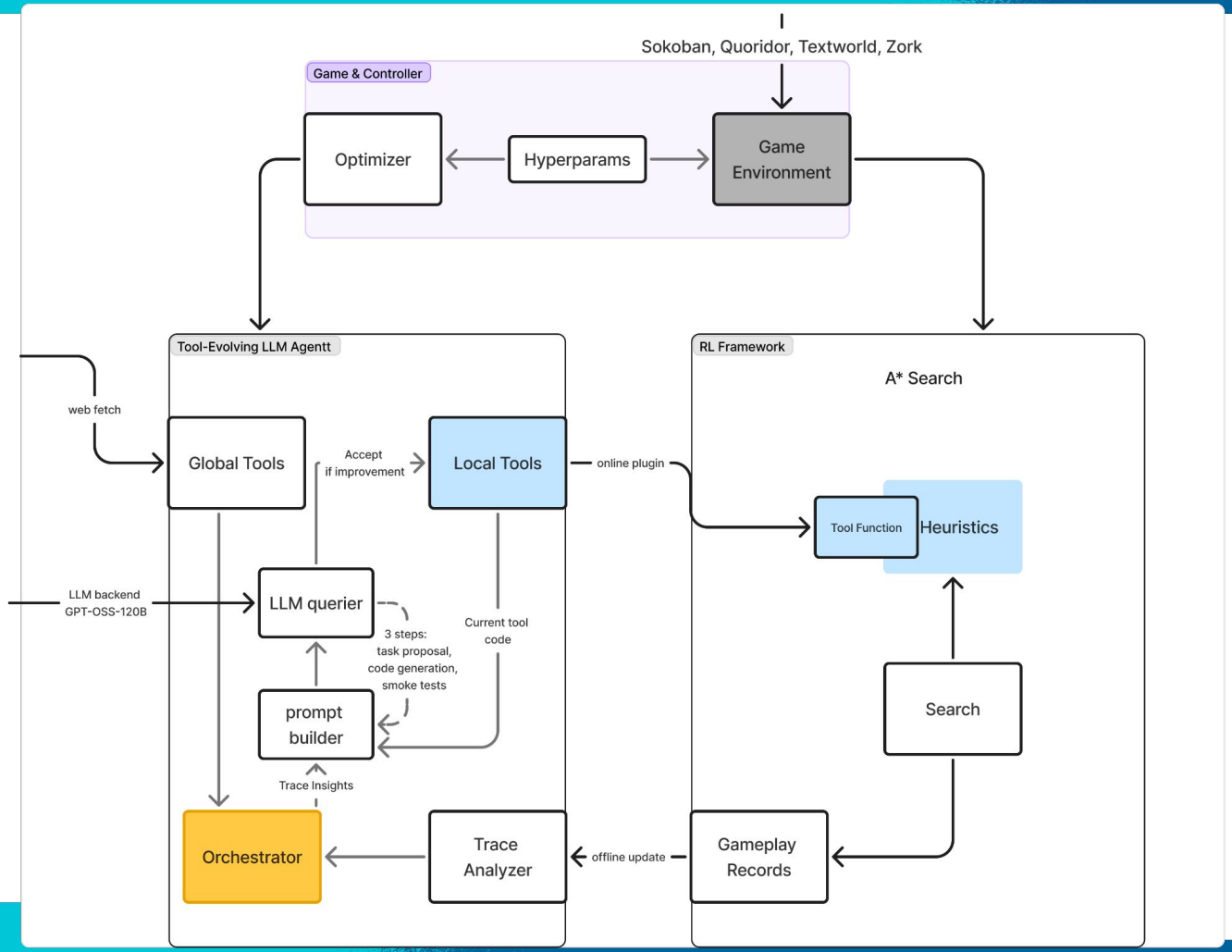
Architecture



UC San Diego

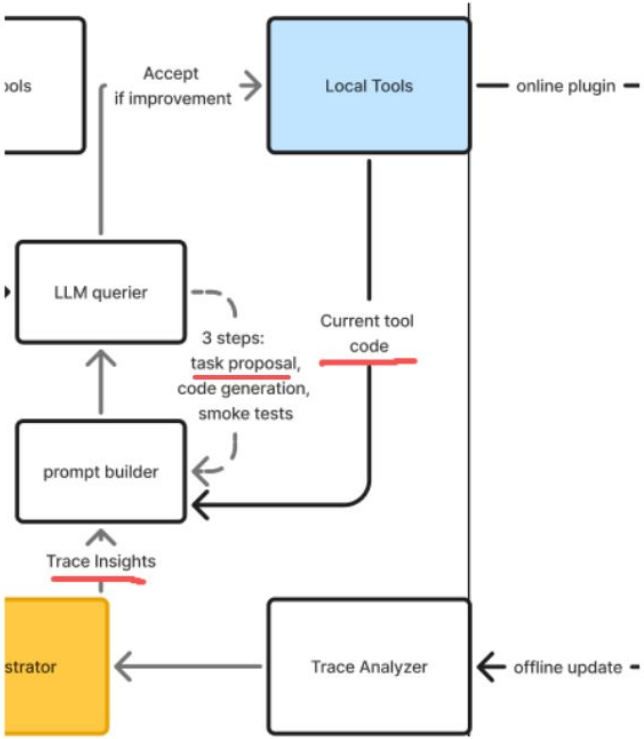
- RL Framework not limited to MCTS
- Modualize optimizable sections, allowing LLM creation & modification
- Adaptable to all turn-based games
- Codebase: https://github.com/zhuyuezx/CSE291A_Project

Self-Evolving Game Agent(A*)



- A* suitable for games with simple action space and easy-to-get heuristics but complex, irreversible state space (e.g. Sokoban)
- MCTS suitable for games with complex action space and hard-to-get heuristics (e.g. go, quoridor)

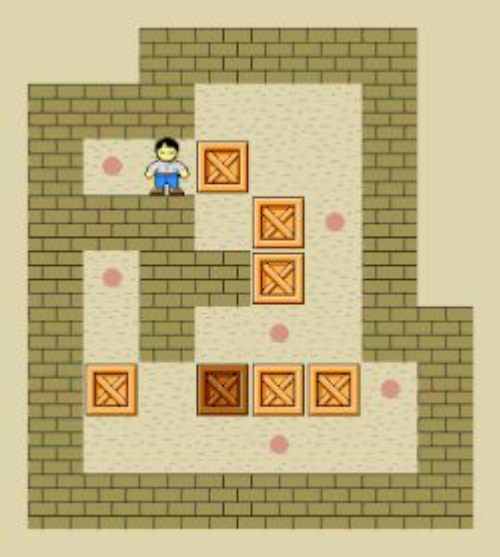
Task Proposal Loop



- Input : Current tool code + Environment feedback given current tools.
- **Mutable source** for improvement seed
- Improvement ideas generated **iteratively**
- Verified and **tested** before incorporated
- Task proposal quality is the **most important fault point**.

Experiments

Sokoban (A* optimization)



```
#####
###   #
#.@$  #
# # $.#
#.# $ #
# # . #
#$ *$$ .#
#   .   #
#########
```

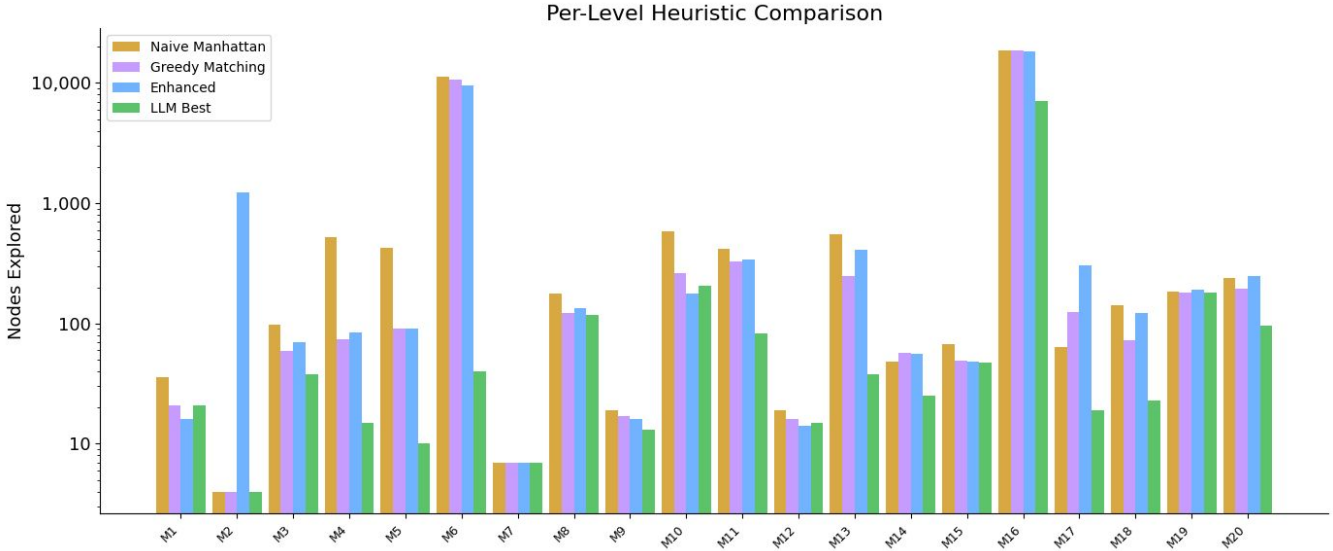
Heuristic Example: **Naive Manhattan Distance**

```
def naive_manhattan(board, boxes):
    """Sum of Manhattan distances"""
    total = 0
    for bx in boxes:
        total += min(bx.manhattan(g) for g in board.goals)
    return total
```

Training on 20 Sokoban Levels

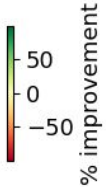
Baseline: Greedy matching + mobility penalty

Hyperparams: 10 iterations, maximum 30,000 search steps

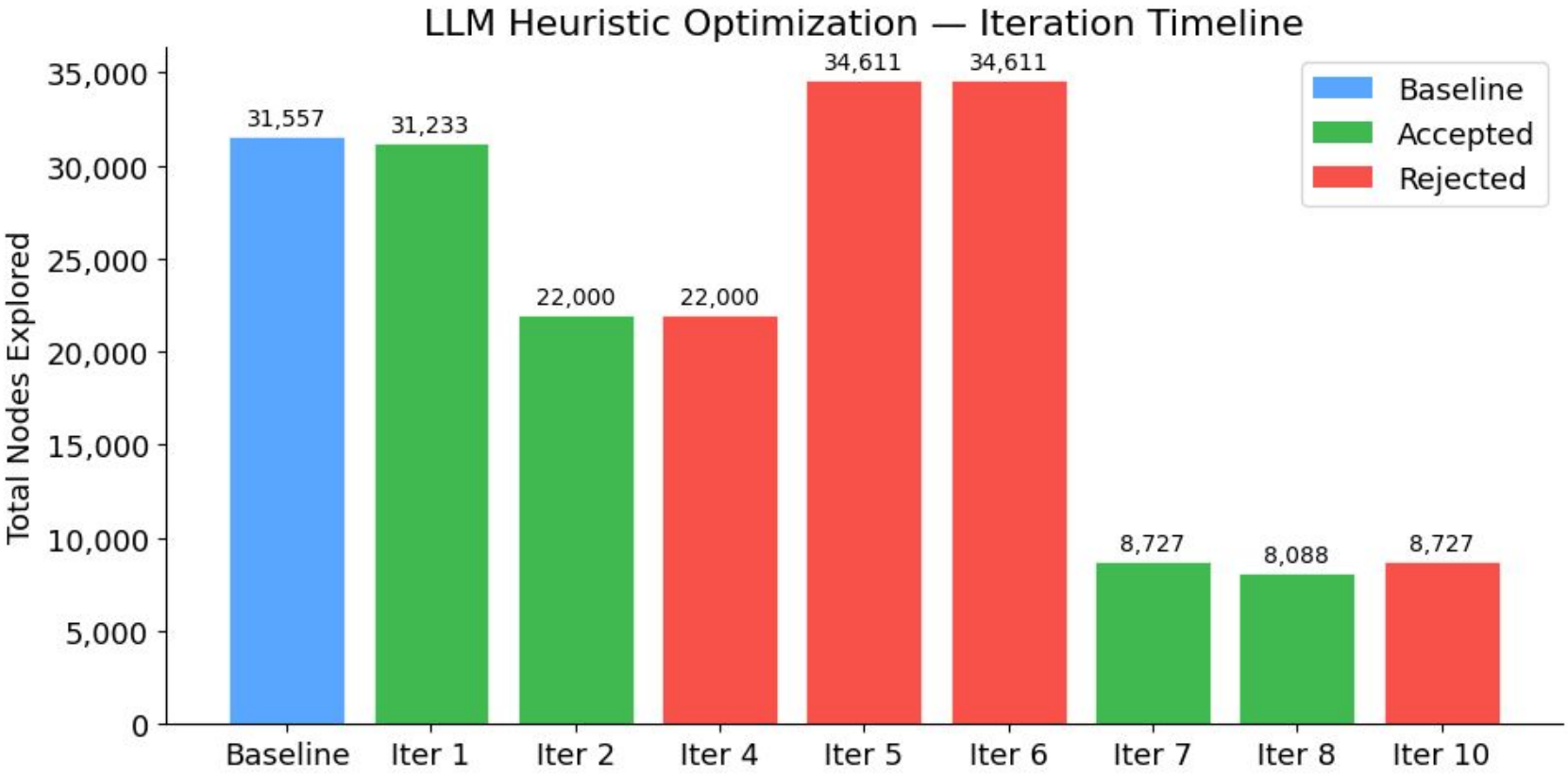


Node Reduction: LLM Best vs Enhanced Baseline (%)

M1 -31%	M2 +100%	M3 +46%	M4 +82%	M5 +89%	M6 +100%	M7 +0%	M8 +12%	M9 +19%	M10 -15%
M11 +76%	M12 -7%	M13 +91%	M14 +55%	M15 +2%	M16 +62%	M17 +94%	M18 +81%	M19 +6%	M20 +62%



Training on 20 Sokoban Levels



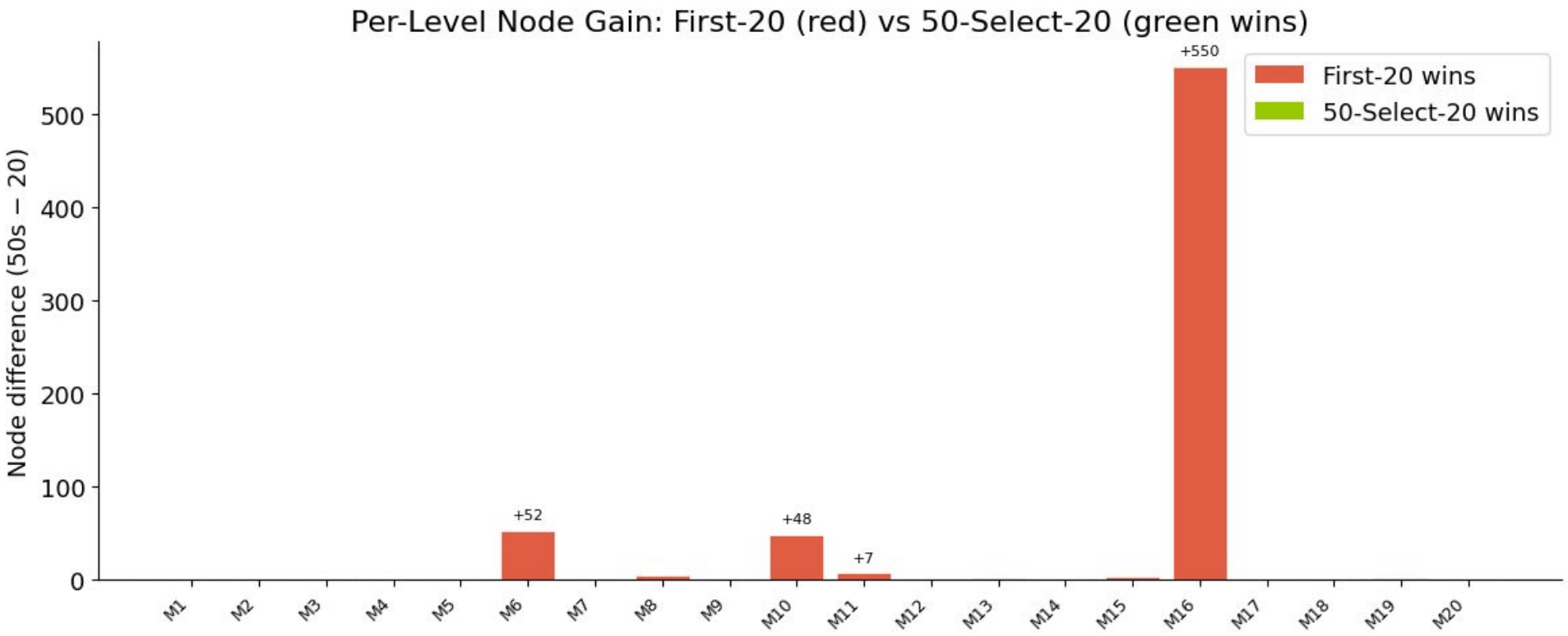
Microban 8	177	122	134	118	↓11.9%
Microban 9	19	17	16	13	↓18.8%
Microban 10	586	263	178	205	↑15.2%
Microban 11	418	325	342	82	↓76.0%
Microban 12	19	16	14	15	↑7.1%
Microban 13	553	247	411	38	↓90.8%
Microban 14	48	57	56	25	↓55.4%
Microban 15	67	49	48	47	↓2.1%
Microban 16	18,618	18,656	18,438	7,091	↓61.5%
Microban 17	64	124	304	19	↓93.8%
Microban 18	142	72	122	23	↓81.1%
Microban 19	185	182	192	181	↓5.7%
Microban 20	241	196	250	96	↓61.6%
TOTAL	**33,479**	**31,268**	**31,557**	**8,088**	**↓74.4%**

Heuristic Source Code

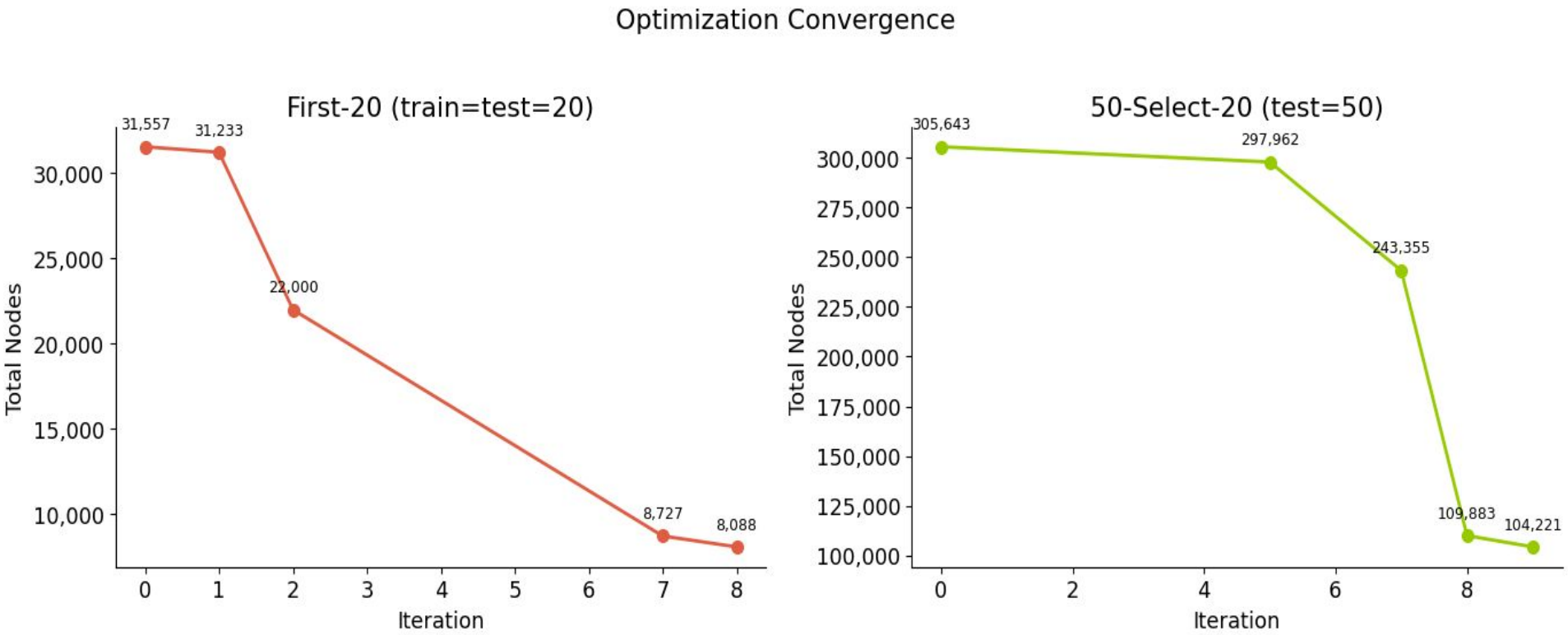
Naive Manhattan | Greedy Matching | Enhanced | llm_iter_1 ✓ | llm_iter_2 ✓ | llm_iter_4 ✗ | llm_iter_5 ✗ | llm_iter_6 ✗ | llm_iter_7 ✓ | llm_iter_8 ✓ | llm_iter_10 ✗

```
def naive_manhattan(board, boxes):
    total = 0
    for bx in boxes:
        total += min(bx.manhattan(g) for g in board.goals)
    return total
```

Interesting: 20 VS 50-select-20

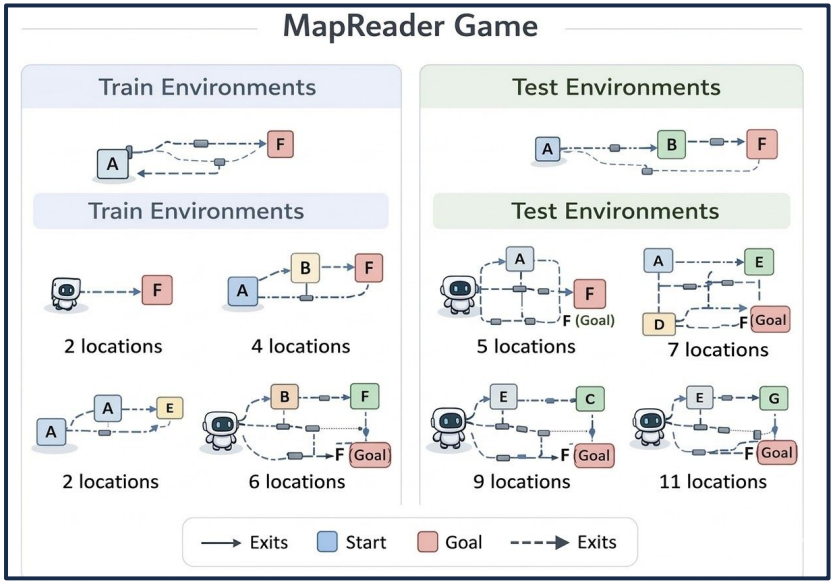


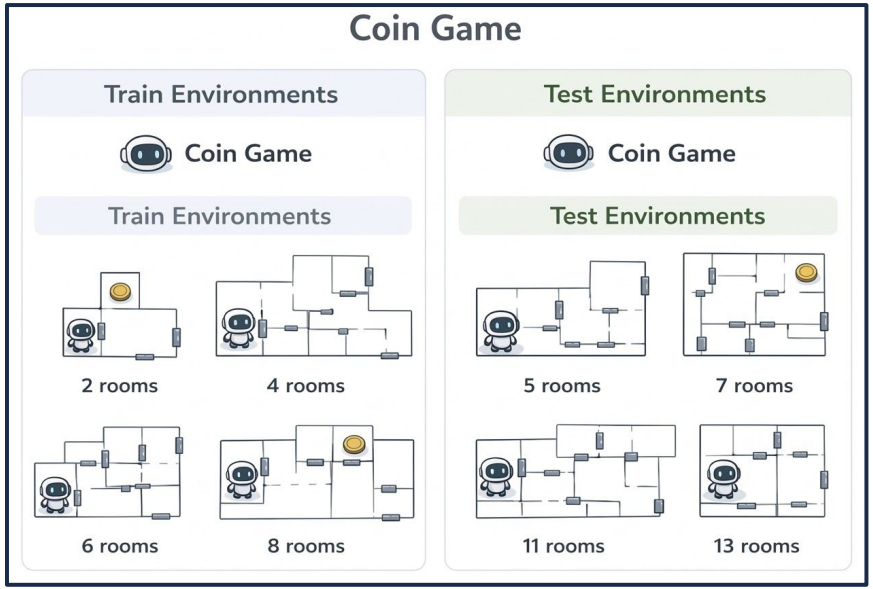
Interesting: 20 VS 50-select-20



Experiment Setup: TextWorld Benchmark

1. Introduce the two games: **Coin** and **MapReader**
2. Use **train environments** to collect gameplay traces
3. Select the **3 hardest/failure cases** for LLM heuristic optimization
4. Evaluate on **larger, unseen test environments**



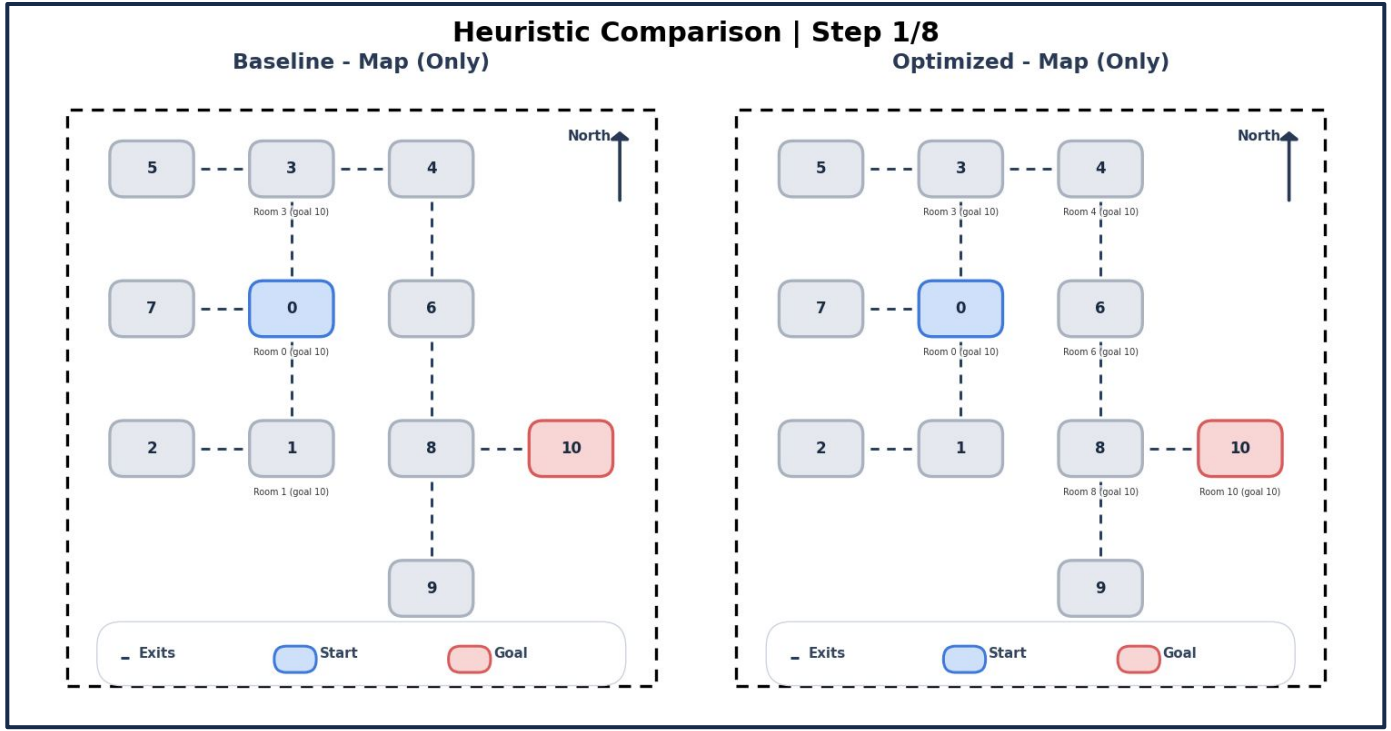


TextWorld Benchmark: Behavior Example

1. Baseline heuristic v.s LLM optimized heuristics for **4 phase**.

2. Base heuristic gets **stuck** in **local** loops

3. Optimized heuristic encourages **global** exploration



TextWorld Benchmark: Optimization Results

UC San Diego

Heuristic difference: **Simulation**

- Baseline: **uniform** random rollout
- Optimized: score-guided rollout
- Rewards task actions and goal progress

Simulation optimization improves test correctness -> generalized heuristic

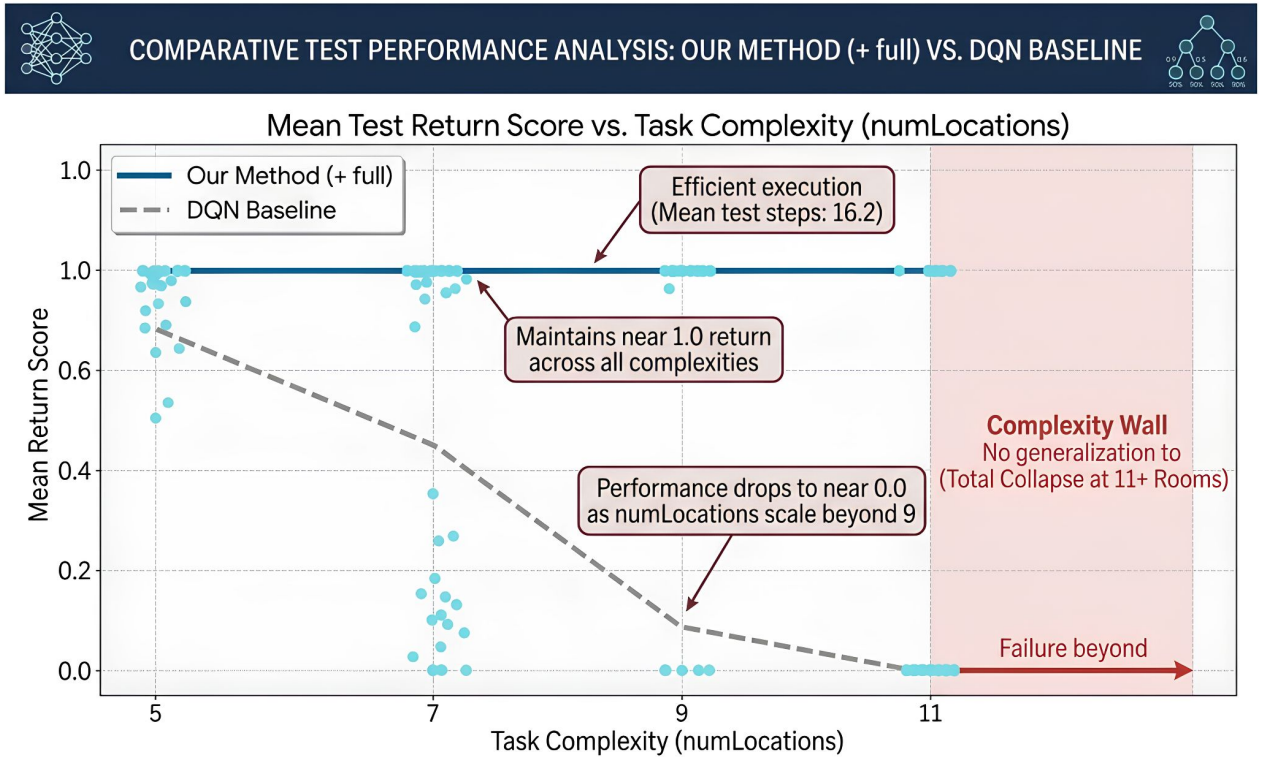
```
# Baseline Simulation

while not terminal:
    a ← random(legal_actions)
    state ← apply(state, a)
```

```
# Optimized Simulation

while not terminal:
    for a in legal_actions:
        score(a) ← heuristic_score(a, state)

    a ← random(argmax score(a))
    state ← apply(state, a)
```

Table 1: Results across configurations on train/test splits. Correct is #correct/#total; Δ is the difference in Correct vs. the baseline on the same split; Ret and Steps are the mean return and mean steps.

Configuration	Train Correct	Train Δ	Train Ret	Train Steps	Test Correct	Test Δ	Test Ret	Test Steps
baseline	62/72	0	0.861	9.9	55/90	0	0.606	14.4
+ selection	63/72	1	0.875	12.8	75/90	20	0.833	15.8
+ expansion	62/72	0	0.819	10.9	55/90	0	0.478	11.3
+ simulation	**72/72**	**10**	**1.000**	41.0	**90/90**	**35**	**1.000**	45.1
+ backpropagation	64/72	2	0.889	**8.5**	60/90	5	0.656	**10.2**
+ full	70/72	8	0.972	14.9	89/90	34	0.989	16.2

Experiment - heuristic generalizability



llm-MCTS pass:
60~90(full)/90

DQN pass rate:
27~33(opt)/90
(test blind - not trained on test env)

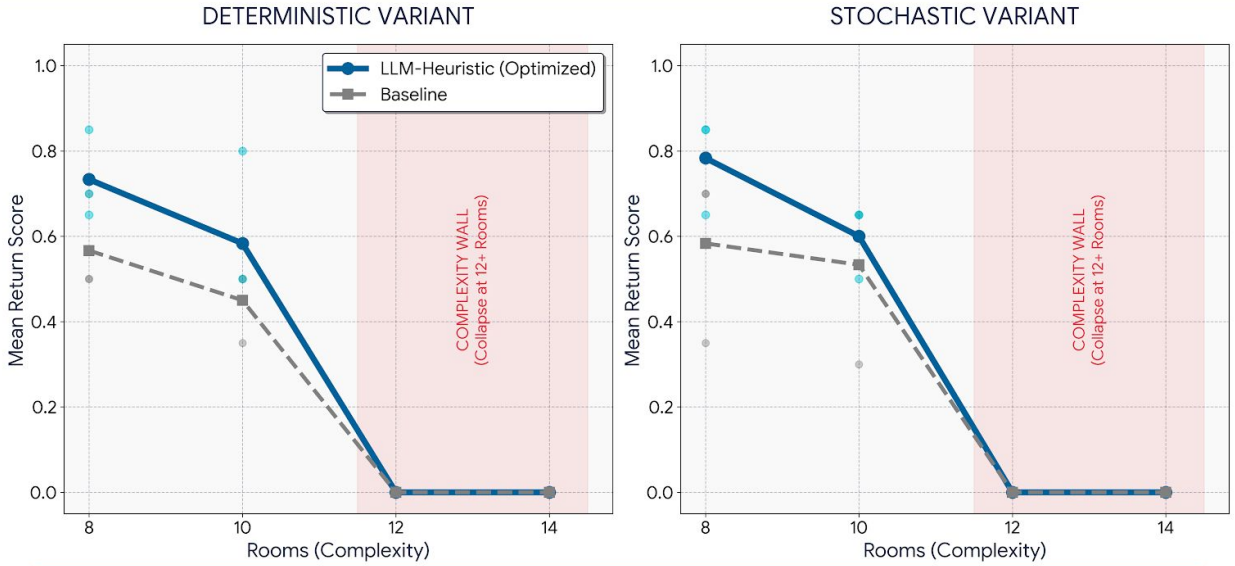
w/:
boltzmann exploration
success replay buffer
stagnation threshold
early stopping
step cost penalty
……

Experiment - with Zork

- Up to 22 interconnected rooms (vs simple grid), with surface + underground layers
- 6 treasures to collect AND deposit back at base (vs pick up 1 coin)
- Multi-step dependencies:
  get lantern → light it → go underground → kill troll → access treasures → return to deposit
- Instant death states: entering dark rooms without lantern
- Resource management: limited lantern fuel
- State space orders of magnitude larger
  (room × inventory × items × fuel × troll × lantern × trophy case)

Experiment - with Zork

PERFORMANCE ANALYSIS: LLM-HEURISTIC MCTS VS. BASELINE



KEY TAKEAWAY: The Optimized MCTS algorithm provides a ~20% performance lift in solvable environments (8-10 rooms), but is limited by the "Complexity Wall" starting at 12 rooms where state-space dependencies peak.

Limitations

- We were not able to meaningfully evaluate **long-horizon behavior** of tool quality
  - After ~1000 code iterations, does the tool quality collapse, plateau, or see eureka moments of continuous improvement?
- Iterative task proposal can get **stuck in parameter tuning** without proposing new ideas. Task proposal could use more research.
- MCTS heuristic tools show some **transferability** on same game different setups
  - Lacks investigation on cross game transferrable tools
- Evaluation limited to **simple environments**
  - Agent demonstrated improvements on multiple simple games, lacked a larger, more impressive game environment to evaluate high impact improvements.

Future Work

- **Border Agent Scope**
  - We limited LLM agent's scope of read/write access to only specific tool files to specific RL frameworks (MCTS) for **feasible evaluation**.
  - The LLM agent might perform better with **elevated scope** - access to full tool folders, freedom to switch RL frameworks that interact with environment, or freedom to modify its own prompt engine/LLM workflow .
- **Task proposal** effectiveness.
- **Generalizing** beyond white-box games
  - The game environments we evaluated are **whitebox** with explicit code endpoint for interaction.
  - Potentially allow LLM agent to figure out/implement interaction frameworks with **blackbox games**. Such as using screenshots to learn game interaction but implement tools to later build encapsulated movesets with mouse clicks.

References

- C. B. Browne et al., "A Survey of Monte Carlo Tree Search Methods," in IEEE Transactions on Computational Intelligence and AI in Games, vol. 4, no. 1, pp. 1-43, March 2012, doi: 10.1109/TCIAIG.2012.2186810. keywords: {Games;Monte Carlo methods;Artificial intelligence;Game theory;Computers;Markov processes;Decision theory;Artificial intelligence (AI);bandit-based methods;computer Go;game search;Monte Carlo tree search (MCTS);upper confidence bounds (UCB);upper confidence bounds for trees (UCT)},
- Shi, Zijing, Meng Fang, and Ling Chen. Monte Carlo Planning with Large Language Model for Text-Based Game Agents. arXiv:2504.16855, 2025.
- Li, Haotian, et al. Yunjue Agent Tech Report: A Fully Reproducible, Zero-Start In-Situ Self-Evolving Agent System for Open-Ended Tasks. arXiv:2601.18226, 2026.

UC San Diego

THANK YOU!